

LAB 1

Game Data Analyzer

Duration: 120 minutes*A gentle start with C++ vectors, simple algorithms and your first template function*

Today's aim: Build confidence. You will start with simple game data, get working output quickly, and then improve the same program step by step. You are not expected to know templates before starting.

What you should be able to do by the end

- Create and use a `std::vector` to store game-related values.
- Loop through a vector and process its values.
- Write a simple algorithm to find the largest value.
- Move repeated logic into a function.
- Explain why one function written only for `int` cannot directly handle float data.
- Use a basic C++ template so the same algorithm works with more than one data type.

Suggested pace

Time	What we will do
0-10 min	Check your C++ project and run a small program
10-30 min	Work with player scores
30-50 min	Find the highest score
50-65 min	Add enemy movement speeds
65-85 min	Convert <code>largest()</code> into a template
85-95 min	Your <code>smallest()</code> challenge
95-100 min	Quick recap / exit check

How to work: Type the code yourself. Run it often. Change one thing at a time. If something breaks, read the compiler message before changing more code. Getting an error is part of programming.

Activity 0 - Make Sure Your Program Runs

Before working with game data, make sure your C++ setup is working. Create or open a console project and put the following in main.cpp:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello DSA Gaming!" << endl;
    return 0;
}
```

Checkpoint: Run the program. If you see “Hello DSA Gaming!” in the terminal/console, you are ready to continue.

If you are using VS Code, a typical C++20 compile command is:

```
Windows: g++ -std=c++20 main.cpp -o main
         .\main.exe

macOS:    clang++ -std=c++20 main.cpp -o main
         ./main
```

Activity 1 - Meet the Game Data

Imagine these are the scores from five rounds of a game:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> scores = {120, 450, 230, 900, 340};

    return 0;
}
```

What this means: `vector<int>` stores a collection of integer values. The vector is called `scores` and it currently contains five game scores.

Task 1A - Display every score

Add this loop inside `main()`:

```
for (int score : scores)
{
```

```
    cout << score << endl;  
}
```

Expected output: 120, 450, 230, 900, 340 (one value per line).

Task 1B - Calculate the total score

- Create an integer variable called total and start it at 0.
- Loop through scores again.
- Add each score to total.
- Print the final value.

Predict first: What total do you expect before running the program? Write your prediction, then check it with C++.

Task 1C - Count the high scores

How many scores are greater than 300?

- Create count and start it at 0.
- For every score, check whether score > 300.
- If the condition is true, increase count by 1.
- Display count at the end.

Checkpoint: Your program should now answer: What values are stored? What is their total? How many are above 300?

Activity 2 - Turn the Idea into an Algorithm

Now let us find the highest score. Before coding, read the steps:

1. Assume the first score is the largest so far.
2. Look at each score in the vector.
3. If the current score is larger, replace largestValue.
4. Continue until every value has been checked.
5. Return the largest value.

Start with an int version

```
int largest(vector<int> values)  
{  
    int largestValue = values[0];  
  
    for (int value : values)  
    {  
        if (value > largestValue)  
        {  
            largestValue = value;  
        }  
    }  
}
```

```
    return largestValue;
}
```

Place the function above main(), then call it from main():

```
cout << "Highest score: " << largest(scores) << endl;
```

Connect to the lecture: The vector is the data structure. The step-by-step search inside largest() is the algorithm. Together they solve the game problem.

Activity 3 - A New Game Requirement

Our game now needs to analyse enemy movement speeds. Add this vector inside main():

```
vector<float> speeds = {3.5f, 7.2f, 4.8f, 9.1f};
```

Now try this line:

```
cout << "Fastest speed: " << largest(speeds) << endl;
```

What should happen: You should get a compiler error. That is expected. Our largest() function was written for vector<int>, but speeds is vector<float>.

We could copy the whole function and replace every int with float, but then we would have two versions of the same algorithm. That is unnecessary repetition.

The problem: Same algorithm + different data type = repeated code. This is exactly where a C++ template becomes useful.

Activity 4 - Your First C++ Template

Replace the int-only largest() function with this generic version:

```
template <typename T>
T largest(vector<T> values)
{
    T largestValue = values[0];

    for (T value : values)
    {
        if (value > largestValue)
        {
            largestValue = value;
        }
    }
}
```

```
    return largestValue;
}
```

Only one new idea: T is a placeholder for a data type. When largest(scores) is called, T becomes int. When largest(speeds) is called, T becomes float.

Now test both calls:

```
cout << "Highest score: " << largest(scores) << endl;
cout << "Fastest speed: " << largest(speeds) << endl;
```

Expected results: Highest score = 900, Fastest speed = 9.1.

Checkpoint: One function is now handling two different data types. This is the main idea you need to understand from today.

Activity 5 - Your Challenge: smallest()

Now you take control. Write a template function that returns the smallest value in a vector:

```
template <typename T>
T smallest(vector<T> values)
{
    // Your code here
}
```

- Test it with scores.
- Test it again with speeds.
- Think: what single comparison changes between largest() and smallest()?

Hint: Start with values[0] as the smallest value. Visit each item and replace smallestValue when you find a smaller one.

Expected answers: smallest score = 120; slowest speed = 3.5.

Bonus - only if you finish early

Write a generic function that returns the total of all values in a vector:

```
template <typename T>
T total(vector<T> values)
{
    // Your code here
}
```

Do not worry if you do not reach the bonus. It is only extra practice.

Before You Leave - 5 Minute Exit Check

1. What does `vector<int>` mean?
2. In `largest()`, what is the algorithm doing?
3. Why did `largest(vector<int>)` fail when we passed `vector<float>`?
4. What does `T` represent in `template <typename T>`?
5. In one sentence: why are templates useful?

You have achieved the goal if: You can run the program, explain how `largest()` works, and understand why the template version can be reused for both scores and speeds. The next two-hour lab will build on this and introduce more challenging tasks.